Ok. Con la soluzione Perl stiamo comunque utilizzando templates yaml.j2 ?

No: con la soluzione Perl **non useresti** `.yaml.j2`, che è una convenzione tipica di Jinja2. Useresti template **Template::Toolkit**, normalmente con estensione `.tt` o `.tt2`; il contenuto può comunque essere YAML, ma la sintassi dei segnaposto cambia. Jinja e Template::Toolkit sono entrambi motori di templating testuale e possono generare YAML, ma non condividono la stessa sintassi. [1] [2]

Convenzione consigliata

Per evitare ambiguità, usa estensioni che dichiarino il motore:

```
Python + Jinja2:
services/postgres/templates/compose.yaml.j2

Perl + Template::Toolkit:
services/postgres/templates/compose.yaml.tt
```

Oppure, ancora più esplicito:

```
services/postgres/templates/compose.yaml.tt2
```

L'estensione non è tecnicamente obbligatoria: Jinja può caricare file con qualsiasi suffisso e Template::Toolkit processa file di testo arbitrari. Ma `.j2` e `.tt` comunicano immediatamente quale motore è richiesto. [3] [1]

Differenza di sintassi

Jinja2

```
services:
  {{ logical_name }}:
    image: {{ image }}
{% if host_port %}
    ports:
      - "127.0.0.1:{{ host_port }}:{{ container_port }}"
{% endif %}
{% for key, value in environment.items() %}
      {{ key }}: {{ value }}
{% endfor %}
```

Jinja usa `{{ ... }}` per le espressioni e `{% ... %}` per statement come `if` e `for`.[1]

Template::Toolkit

```
services:
  [% logical_name %]:
    image: [% image %]
[% IF host_port %]
    ports:
      - "127.0.0.1:[% host_port %]:[% container_port %]"
[% END %]
[% FOREACH pair IN environment.pairs %]
      [% pair.key %]: [% pair.value %]
[% END %]
```

Template::Toolkit usa `[% ... %]` per direttive e variabili; per iterare un hash si usa tipicamente `.pairs`, che produce coppie `key`/`value`.[2] [3]

Template PostgreSQL in TT

```
# services/postgres/templates/compose.yaml.tt
services:
  [% logical_name %]:
    image: [% image %]
    profiles:
      - [% profile %]
    restart: unless-stopped
[% IF host_port %]
    ports:
      - "127.0.0.1:[% host_port %]:[% container_port %]"
[% END %]
[% IF data_volume %]
    volumes:
      - [% data_volume %]:[% data_mount %]
[% END %]
    environment:
[% FOREACH pair IN environment.pairs %]
      [% pair.key %]: [% pair.value %]
[% END %]
    healthcheck:
      test: [% healthcheck_test %]
      interval: [% healthcheck_interval %]
      timeout: [% healthcheck_timeout %]
      retries: [% healthcheck_retries %]
```

Nel codice Perl passeresti le stesse variabili che nell'esempio Python:

```
my $output;
$tt->process(
    $template_path,
    {
        logical_name         => $logical_name,
```

```
        image                => $image,
        profile              => $service_type,
        host_port            => $host_port,
        container_port       => $manifest->{container_port},
        data_volume          => $data_volume,
        data_mount           => $volume_spec->{mount},
        environment          => $manifest->{environment} // {},
        healthcheck_test     => $manifest->{healthcheck}{test},
        healthcheck_interval => $manifest->{healthcheck}{interval} // '5s',
        healthcheck_timeout  => $manifest->{healthcheck}{timeout} // '3s',
        healthcheck_retries  => $manifest->{healthcheck}{retries} // 10,
    },
    \$output,
) or die $tt->error;
```

Regola pratica per `docker-dev`

Componente	Python	Perl
Manifest	`service.yaml`	`service.yaml`
Parser YAML	`yaml.safe_load`	`YAML::XS::LoadFile`, con `LoadBlessed` disabilitato
Template	`compose.yaml.j2`	`compose.yaml.tt`
Motore	Jinja2	Template::Toolkit
Delimitatori	`{{ }}`, `{% %}`	`[% %]`
Output	`compose.services.yaml`	`compose.services.yaml`

Quindi: **sì, anche in Perl usi template per generare YAML**, ma non `.yaml.j2`. Rinominerei i template in `.yaml.tt` e manterrei la struttura `services/<type>/templates/`; così un eventuale passaggio da Perl a Python richiede solo la conversione della sintassi dei template, non una riprogettazione dell'architettura. [3] [1]

⁂

1. https://jinja.palletsprojects.com/en/stable/templates/
2. https://template.mailkit.com/manual/Intro.php
3. https://manpages.debian.org/bookworm/libtemplate-perl/Template::Toolkit.3pm
4. https://python-web.teclado.com/section09/lectures/02_new_jinja2_variables/
5. https://github.com/NirDiamant/Prompt_Engineering/blob/main/all_prompt_engineering_techniques/prompt-templates-variables-jinja2.ipynb
6. https://metacpan.org/dist/Template-Toolkit/view/lib/Template/Modules.pod
7. https://metacpan.org/release/ABW/Template-Toolkit-2.14
8. https://metacpan.org/pod/CGI::Application::Plugin::TT
9. https://metacpan.org/release/ABW/Template-Toolkit-2.20
10. https://documentation.bloomreach.com/engagement/docs/jinja-syntax
11. https://jinja.palletsprojects.com/en/stable/api/

12. https://jinja.palletsprojects.com/

13. https://template.mailkit.com/manual/Filters.php

14. https://stackoverflow.com/questions/51091139/current-template-file-name-with-templatetoolkit

15. https://www.geeksforgeeks.org/python/placeholders-in-jinja2-template-python/